LAPORAN PRAKTIKUM PEKAN 5

PEMROGRAMAN BERORIENTASI OBJEK

TRANSAKSI DINAMIS (POLYMORPHISM)

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Nurfiah,S.ST., M.Kom.

Asisten Praktikum:

Ezza Addini

(2311532001)

Link Repository: https://github.com/syasyahalwa22/PrakPBO_B_2511531018

Link Youtube: https://youtu.be/Md7GogNBWAk



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Pemrograman Berorientasi Objek, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Nurfiah,S.ST., M.Kom. selaku dosen mata kuliah Pemrograman Berorientasi Objek yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Oktober 2026

Penulis

BAB I PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek merupakan salah sastu konsep penting dalam pemrograman yang digunakan untuk membuat program berdasarkan objek tertentu. Dengan ini, suatu permasalahan dapat diselesaikan secara terstruktur, karena dapat menyelesaikan masalah tersebut dengan mengelompokkan objek-objek tersebut. Konsep dasar PBO meliputi class, object, constructor, atribut, dan method. Class berfungsi sebagai rancangan untuk membuat object, sedangkan object merupakan bentuk nyata dari class. Atribut digunakan untuk menyimpan data atau keadaan object, sementara method digunakan untuk mendefinisikan tindakan atau perilaku yang dapat dilakukan oleh object. Constructor digunakan untuk memberikan nilai awal ketika sebuah object dibuat.

1.2 Tujuan

1.2.1 Meningkatkan pemahaman tentang dasar-dasar bahasa Java dalam PBO.

1.2.2 Dapat membuat sebuah program dengan menggunakan atribut-atribut yang akan digunakan.

1.2.3 Melatih kemampuan logika pemrograman.

1.3 Manfaat Praktikum

1.3.1 Menambah pemahaman tentang dasar-dasar bahasa Java dalam PBO

1.3.2 Melatih kemampuan membuat program menggunakan bahasa Java.

1.3.3 Memahami proses pembuatan program pada Java.

BAB II PEMBAHASAN

DASAR TEORI

Program sistem perbankan mini ini menggunakan konsep Pemrograman Berorientasi Objek dalam Java, yaitu program dibagi menjadi beberapa class seperti Rekening dan Transaksi. Class Rekening digunakan untuk mengolah data rekening dan transaksi, sedangkan class Transaksi menyimpan detail dalam transaksi. Data transaksi disimpan menggunakan ArrayList riwayatTransaksi sehingga setiap transaksi yang berhasil dapat dicatat dan ditampilkan. Pada proses tarik tunai, digunakan if-else untuk mengecek kecukupan saldo sebelum transaksi dicatat sebagai Debit. Kemudian, method cekMutasi() menggunakan perulangan for-each untuk mengambil setiap transaksi dalam riwayatTransaksi dan menampilkan detailnya melalui method cetakDetail(). Dengan begitu, program dapat mencatat sekaligus menampilkan riwayat transaksi dari rekening yang sedang aktif.

2.1 Langkah-Langkah Praktkum

2.1.1 Kode Program Rekening

```
package pekan5;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
            this.nomorRekening = nomor;
            this.namaPemilik = nama;
            this.saldo = saldoAwal;

            if (pinAwal.length() == 6) {
                    this.pin = pinAwal;
            } else {
                    System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
                    this.pin = "123456";
            }

            this.riwayatTransaksi = new ArrayList<>();
            System.out.println("Rekening atas nama " + namaPemilik +
                            " berhasil dibuat dengan saldo Rp" + saldo);
    }
```

```
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    public boolean otentikasi(String inputPin) {
            return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
            if (nominal > 0) {
                    saldo += nominal;
                    String idTrx = "TRX-S-" + System.currentTimeMillis();
                    Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
                    riwayatTransaksi.add(trxBaru);

                    System.out.println("Setor tunai Rp" + nominal +
                                    " berhasil. Saldo saat ini: Rp" + saldo);
            } else {
                    System.out.println("Gagal: Nominal setor harus lebih dari 0!");
            }
    }

    public void cekInformasi() {
            System.out.println("--- INFO REKENING ---");
            System.out.println("No. Rekening : " + nomorRekening);
            System.out.println("Nama Pemilik : " + namaPemilik);
            System.out.println("Saldo Akhir  : Rp" + saldo);
            System.out.println("----------------------");
    }

    public void cekMutasi() {
            System.out.println("\n--- MUTASI REKENING ---");

            if (riwayatTransaksi.isEmpty()) {
                    System.out.println("Belum ada transaksi pada rekening ini");
            } else {
                    for (Transaksi transaksi : riwayatTransaksi) {
                            transaksi.cetakDetail();
                    }
            }

            System.out.println("-----------------------");
    }

    public boolean tarikTunai(double nominal) {
            if (nominal <= 0) {
                    System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
                    return false;
            } else if (nominal > saldo) {
                    System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp."
                                                                    + saldo );
                    return false;
            } else {
                    saldo -= nominal;

                    String idTrx = "TRX-T-" + System.currentTimeMillis();
                    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

                    System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Sisa saldo: Rp" + saldo);
                    return true;
            }

    }

    public void gantiAkun(String nomor, String nama, double saldoAwal) {
            nomorRekening = nomor;
```

```
            namaPemilik = nama;
            saldo = saldoAwal;
            System.out.println("Rekening atas nama " + namaPemilik +
                               " berhasil diganti saldo Rp " + saldo);

            }

    }
```

```
private String nomorRekening;
        private String namaPemilik;
        private String pin;

        protected double saldo;
        protected ArrayList<Transaksi> riwayatTransaksi;
```

kode tersebut digunakan untuk menyimpan data sebuah rekening. Private berarti hanya bisa diakses dalam class tersebut, sedangkan protected bisa diakses oleh class tersebut dan turunannya.

```
public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    if (pinAwal.length() == 6) {
        this.pin = pinAwal;
    } else {
        System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
        this.pin = "123456";
```

Constructor ini dijalankan ketika objek rekening baru dibuat. Data nomor rekening, nama pemilik, saldo awal, dan PIN diberikan melalui parameter. Lalu melakukan pengecekan panjang PIN, jika PIN terdiri dari 6 karakter, PIN tersebut digunakan. Jika tidak, siste meberikan peringatan dan menggunakan PIN default 123456.

```
this.riwayatTransaksi = new ArrayList<>();
System.out.println("Rekening atas nama " + namaPemilik +
        " berhasil dibuat dengan saldo Rp" + saldo);
```

Membuat ArrayList baru yang awalnya masih kosong untuk menyimpan transaksi rekening.

```
public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik() { return namaPemilik; }

public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
```

Menggunakan method getter untuk mengambil nomor rekening dan nama pemilik. Karena atributnya menggunakan private, data tersebut tidak bisa diakses langsung dari class lain, sehingga diperlukan getter.

Lalu, mengecek apakah PIN yang dimasukkan pengguna sesuai dengan PIN rekening. Hasilnya berupa true jika benar dan false jika salah.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal +
                " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
```

Digunakan utnuk melakukan penyetoran uang. Jika nominal lebih dari 0, uang akan ditambahkan ke saldo. Setelah setor berhasil, sistem membuat ID transaksi, membuat objek Transaksi, lalu memasukkannya ke dalam riwayatTransaksi.

```
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
    System.out.println("----------------------");
}
```

Method ini digunakan untuk menampilkan informasi rekening seperti nomor rekening, nama pemilik, dan saldo terakhir.

```
public void cekMutasi() {
    System.out.println("\n--- MUTASI REKENING ---");

    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }

    System.out.println("----------------------");
```

Method cekMutasi digunakan untuk menampilkan seluruh riwayat transaksi, lalu mengecek apakah belum ada transaksi. Jika ada transaksi, maka perulangan digunakan untuk mengambil setiap transaksi dalam riwayatTransaksi, kemudian memanggil cekDetail() untuk menampilkan detailnya.

```
public void tarikTunai(double nominal) {
    if (nominal > saldo) {
        System.out.print("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp."
                         + saldo );
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Penarikan tunai berhasil!");
        System.out.println("Saldo saat ini: Rp" + saldo);
    }
```

Method ini digunakan untuk melakukan penarikan uang dari rekening. Pertama sistem akan mengecek nominal penarikan lebih besar atau tidak dari saldo. Jika iya, transaksi ditolak karena saldo tidak mencukupi. Jika saldo mencukupi, saldo akan dikurangi sesuai dengan jumlah uang yang ditarik. Kemudian mencatat transaksi menggunakan ID dengan jenis debit dan dimasukkan ke riwayat transaksi.

```
public void gantiAkun(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik +
            " berhasil diganti saldo Rp " + saldo);
```

Method ini digunakan untuk mengganti data rekening yang sedang digunakan. Nomor rekening, nama pemilik, dan saldo akan diganti berdasarkan parameter yang diberikan.

2.1.2 Kode Program mainRekening

```
package pekan5;

import java.util.ArrayList;
import java.util.Scanner;

public class mainRekening {
public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif =null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
        System.out.println("\nMenu Utama:");
```

```
        System.out.println("1. Buka Rekening Baru");
        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Informasi Rekening");
        System.out.println("5. Ganti Akun");
        System.out.println("6. Cek Mutasi (Riwayat)");
        System.out.println("0. Keluar");
        System.out.print("Pilih Menu: ");

        int pilihan = input.nextInt();
        input.nextLine();

        switch (pilihan) {
        case 1:
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal: ");
                double saldo = input.nextDouble();

                akunAktif = new Rekening(no, nama, saldo);
                daftarRekening.add(akunAktif);
                break;

        case 2:
                if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening!");
                } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor);
                }
                break;

        case 3:
                if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening!");
                } else {
                        System.out.print("Masukkan nominal tarik tunai: ");
                        double tarik = input.nextDouble();
                        akunAktif.tarikTunai(tarik);
                }
                break;
```

```
        case 4:
                if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka
rekening!");
                } else {
                        akunAktif.cekInformasi();
                }
                break;

        case 5:
            System.out.println("\n--- GANTI AKUN ---");

            if (daftarRekening.isEmpty()) {
                System.out.println("Belum ada rekening yang tersedia.");
                break;
            }

            System.out.print("Masukkan nomor rekening: ");
            String nomorCari = input.nextLine();

            boolean ditemukan = false;

            for (Rekening rekening : daftarRekening) {

                if (rekening.nomorRekening.equals(nomorCari)) {
                    akunAktif = rekening;
                    ditemukan = true;

                    System.out.println("Berhasil berganti ke rekening atas nama
"
                            + akunAktif.namaPemilik);

                    break;
                }
            }

            if (!ditemukan) {
                System.out.println("Rekening tidak ditemukan.");
            }

            break;

        case 6:
                if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening!");
```

```
            } else {
                akunAktif.cekMutasi();
            }
            break;

        case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima kasih!");
                break;

        default:
                System.out.println("Pilihan tidak valid!");
                }
        }
        input.close();
```

```
public class mainRekening {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif =null;
        boolean isRunning = true;
```

Merupakan bagian utama program yang dimana semua proses sistem akan dijalankan dari method main. Input digunakan untuk membaca input pengguna, daftarRekening untuk menyimpan semua rekening yang sudah dibuat, akunAktif untuk menyimpan rekening yang sedang digunakan, dan isRunning digunakan untuk mengatur apakah program masih berjalan atau sudah berhenti.

```
while (isRunning) {
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cek Mutasi (Riwayat)");
System.out.println("0. Keluar");
System.out.print("Pilih Menu: ");

int pilihan = input.nextInt();
input.nextLine();
```

Perulangan yang membuat menu terus ditampilkan selama isRunning bernilai true. Program baru berhenti apabila pengguna memilih menu 0.

nextInt() digunakan untuk membaca pilihan berupa angka. Setelah itu, nextLine() digunakan untuk membersihkan sisa enter agar input String berikutnya tidak langsung lewat.

```
switch (pilihan) {
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Buat PIN (6 digit): ");
    String pin = input.nextLine();

    akunAktif = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(akunAktif);
    break;
```

Pada menu ini pengguna diminta memasukkan nomor rekening, nama pemilik, saldo awal, dan PIN. Baris pertama membuat objek Rekening baru. Kemudian rekening tersebut dimasukkan ke daftarRekening. Rekening yang baru dibuat juga langsung menjadi akunAktif, sehingga pengguna bisa langsung melakukan transaksi pada rekening tersebut.

```
case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;
```

Pertama sistem mengecek apakah sudah ada rekening aktif, jika belum akan muncul pesan bahwa pengguna belum memiliki rekening. Jika sudah ada, pengguna memasukkan nominal setor, kemudian method setorTunai pada class Rekening dipanggil untuk menambahkan uang ke saldo dan mencatat transaksi.

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput = input.nextLine();

    if (akunAktif.otentikasi(pinYangDiinput)) {
        System.out.print("Masukkan nominal tarik tunai: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);

    akunAktif.tarikTunai(tarik);
    } else {
        System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!"); }
    }
    break;
```

Case 3 digunakan untuk melakukan penarikan uang dari rekening yang sedang aktif. Setelah itu, pengguna diminta memasukkan PIN, lalu method otentikasi() akan membandingkan PIN yang dimasukkan dengan PIN rekening. Jika PIN benar, sistem meminta nominal penarikan dan menjalankan method tarikTunai(), jika PIN salah, transaksi tidak akan dilakukan.

```
case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }

    break;
```

Case 4 digunakan untuk melihat informasi dari rekening yang sedang aktif. Jika akunAktif null maka program tidak dapat memproses informasi pada rekening. Namun jika akunAktif tidak null, program akan menampilkan informasi rekening berupa nomor rekening, nama pemilik, dan saldo rekening.

```
if (!ditemukan) {
    System.out.println("Rekening tidak ditemukan.");
}

break;
```

Case 5 digunakan untuk mengganti rekening yang sedang aktif dengan rekening yang lain yang sudah tersimpan di daftarRekening. Program meinta pengguna memasukkan nomor rekening yang ingin dicari, kemudian menggunakan perulangan for-each untuk memeriksa setiap rekening dalam daftarRekening. Jika nomor rekening yang dicari ditemukan, rekening tersebut disimpan ke akunAktif sehingga menjadi

rekening yang aktif. Variabel ditemukan digunakan untuk mengetahui apakah rekening berhasil ditemukan, dan jika tidak ditemukan program akan menampilkan pesan bahwarekening tidak tersedia.

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            akunAktif.cekMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Case 6 digunakan untuk melihat mutasi atau riwayat transaksi rekening yang sedang aktif. Sebelum mutasi ditampilkan, sistem meminta PIN, jika PIN benar, method cekMutasi pada class Rekening akan menampilkan seluruh transaksi yang tersimpan di riwayatTransaksi. Jika PIN salah, mutasi tidak ditampilkan dan akan memunculkan pesan.

```
 case 0:
     isRunning = false;
     System.out.println("Sistem ditutup. Terima kasih!");
     break;

 default:
     System.out.println("Pilihan tidak valid!");
     }
 }
 input.close();
```

Case 0 digunakan untuk mengakhiri program, dengan menghentikan perulangan while sehingga program berhenti menampilkan menu. Default digunakan untuk menangani menu yang tidak tersedia.

2.1.3 Kode Program Transaksi

```
package pekan5;

public class Transaksi {
private String idTransaksi;
private String jenis;
private double nominal;

public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
```

```
        this.jenis = jenis;
        this.nominal = nominal;
}

public String getIdTransaksi() {return idTransaksi; }
public String getJenis() { return jenis; }
public double getNominal() {return nominal; }


public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
                        " | Nominal: Rp" + nominal);
}
}
```

```
3  public class Transaksi {
4      String idTransaksi;
5      String jenis;
6      double nominal;
```

Membuat class Transaksi yang berfungsi sebagai tempat menyimpan data setiap transaksi yang dilakukan pada rekening, seperti ID transaksi, jenis transaksi, dan nominal transaksi. Ketiga atribut digunakan untuk menyimpan informasi transaksi.

```
8      public Transaksi(String id, String jenis, double nominal) {
9          this.idTransaksi = id;
10         this.jenis = jenis;
11         this.nominal = nominal;
```

Constructor digunakan untuk membuat objek Transaksi ekaligus mengisi data transaksinya. Constructor dipanggil ketika transaksi setor atau tarik dibuat pada class Rekening.

```
public String getIdTransaksi() {return idTransaksi; }
public String getJenis() { return jenis; }
public double getNominal() {return nominal; }
```

Getter digunakan untuk mengambil nilai atribut yang bersifat private.

```
public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
            " | Nominal: Rp" + nominal);
```

Method cekDetail digunakan untuk menampilkan detail transaksi ke layar. Method ini mengambil nilai idTransaksi, jenis, dan nominal, kemudian menggabungkannya menjadi satu baris informasi. Method ini

dipanggil oleh cekMutasi pada class Rekening untuk menampilkan setiap transaksi yang tersimpan di dalam riwayatTransaksi.

2.1.4 Kode Program RekeningTabungan

```
package pekan5;

public class RekeningTabungan extends Rekening {

private double sukuBunga;

public RekeningTabungan(String nomor, String nama, double saldoAwal, String
pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
}

public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" +
nominalBunga);
}

}
```

`public class RekeningTabungan extends Rekening {` `private double sukuBunga;`
Merupakan turunan dari class Rekening dan menyimpan presentase bunga.
`super(nomor, nama, saldoAwal, pinAwal);` `this.sukuBunga = sukuBunga;`
Memanggil constructor dari class Rekening dan menyimpan nilaisuku bunga.

`public void tambahBungaAkhirBulan() {`
`double nominalBunga = saldo * (sukuBunga / 100);`
`saldo += nominalBunga;`

Menghitung dan menambahkan bunga ke saldo, jumlah bunga yang diperoleh dari saldo.

`String idTrx = "TRX-B-" + System.currentTimeMillis();`
`riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));`
 	
`System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);`

Mencatat bunga sebagai transaksi dan menampilkan informasi bunga yang ditambahkan.

2.1.5 Kode Program RekeningGiro

```
package pekan5;

public class RekeningGiro extends Rekening {
private double batasOverdraft;

public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;

}

@Override
public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
                System.out.println("Gagal: Nominal tarik Giro harus lebih dari
0!");
                return false;
        }

        else if (nominal > (saldo + batasOverdraft)) {
                System.out.println("Transaksi Gagal: Melebihi batas Overdraft!
Maksimal penarika: Rp" +
                                (saldo + batasOverdraft));
                return false;
        }
        else {
                saldo -= nominal;

                String idTrx = "TRX-G-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro",
nominal));

                System.out.println("Tarik tunai Giro Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
                return true;
                }
}
public double getBatasOverdraft() { return batasOverdraft;
}
```

```
public class RekeningGiro extends Rekening {
        private double batasOverdraft;
```

Mewarsi atribut dan method dari Rekening dan batasOverdraft menyimpan batas maksimal saldo minus yang diperbolehkan.

```
        public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
                super(nomor, nama, saldoAwal, pinAwal);
                this.batasOverdraft = batasOverdraft;
```

Memanggil constructor dari class Rekening dan menyimpan nilai batas overdraft.

```
  @Override
  public boolean tarikTunai(double nominal) {
      if (nominal <= 0) {
          System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
          return false;
```

Override merupakan method yang menggantikan implementasi method dari classRekening. Dalam program jika nominal 0, maka penarikan ditolak, jika nominal negatif, penarikan juga ditolak. Return false mengakhiri method dan memberi tahu bahwa penarikan gagal.

```
else if (nominal > (saldo + batasOverdraft)) {
    System.out.println("Transaksi Gagal: Melebihi batas Overdraft! Maksimal penarika: Rp" +
            (saldo + batasOverdraft));
    return false;
}
```

program memeriksa batas overdraft, dengan fasilitas yang memungkinkan rekening melakukan penarikan melebihi saldo yang tersedia sampai batas tertentu. Return false menandakan penarikan gagal dan saldo tidak ubah.

```
else {
    saldo -= nominal;

    String idTrx = "TRX-G-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

    System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp"
    + saldo);
    return true;
    }
```

Bagian else dijalankan jika nominal lebih dari nol dan tidak melebihi batas oeverdraft. Operator -= berarti mengurangi nilai variabel dengan nilai tertentu. Lalu membuat ID transaksi dan mencatat transaksi penarikan ke dalam riwayatTransaksi.

```
        public double getBatasOverdraft() {
                return batasOverdraft;
```

Digunakan untuk mengambil nilai batas overdraft.

2.2 Tugas dan Latihan: Fitur Transfer dan Antar Rekening

2.2.1 **Buat method transfer:** Didalam class **Rekening.java,** tambahkan satu method bar dengan nama:

Public void transfer(Rekening tujuan, double nominal, String pinInput)

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer gagal: PIN yang Anda masukkan salah!");
        return;
}

boolean berhasil = this.tarikTunai(nominal);
if (berhasil) {
    tujuan.setorTunai(nominal);
    System.out.println("Transfer berhasil");
} else {
    System.out.println("Transfer gagal!");
}
```

Menambahkan method transfer yang menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro. Pada logika method, dilakukan otentikasi(pinInput) terlebih dahulu, jika false maka akan dibatalkan. Lalu memanggil this.tarikTunai(nominal), jika this Giro maka akan otomatis menggunakan aturan overdraft. Boolean berhasil menyimpan hasil berupa true atau false. Jika bernilai true, program akan menyetor uang ke rekening tujuan menggunakan tujuan.setorTunai(nominal). Jika false, program tidak menjalankan penyetoran.

2.2.2 **Integrasi ke CLI Utama:** Buka **Main.java** dan tambahkan menu **8. Transfer ke Rekening Lain.**

```
while (isRunning) {
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cek Mutasi (Riwayat)");
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
System.out.println("8. Transfer ke Rekening Lain");
System.out.println("0. Keluar");
System.out.print("Pilih Menu: ");
```

Menambahkan menu Transfer ke Rekening Lain agar pengguna dapat menyetor uang kepada pengguna lain.

```
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan nomor rekening tujuan: ");
        String nomorTujuan = input.nextLine();

        Rekening akunTujuan = null;

        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorTujuan)) {
                akunTujuan = rekening;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Rekening tujuan tidak ditemukan!");
        } else if (akunTujuan == akunAktif) {
            System.out.println("Tidak bisa transfer ke rekening sendiri!");
        } else {
            System.out.print("Masukkan nomor Transfer: ");
            double nominal = input.nextDouble();
            input.nextLine();

            System.out.print("Masukkan PIN Anda: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominal, pinTransfer);
        }
    }
    break;
```

```
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
```

Case 8 dijalankan ketika pengguna memilih menu nomor 8 dan akunAktif == null merupakan pemeriksaan apakah ada rekening yang sedang digunakan atau sama dengan null. Jika belum ada, transfer tidak dapat dilakukan.

```
} else {
    System.out.print("Masukkan nomor rekening tujuan: ");
    String nomorTujuan = input.nextLine();

    Rekening akunTujuan = null;
```

Input.nextLine() untuk membaca nomor rekening yang diketik pengguna. nomorTujuan untuk menyimpan nomor rekening yang ingin dituju, dan

Rekening akunTujuan = null; membuat variabel untuk menyimpan objek rekening penerima, yang awalnya belum ditemukan.

```
for (Rekening rekening : daftarRekening) {
    if (rekening.getNomorRekening().equals(nomorTujuan)) {
        akunTujuan = rekening;
        break;
```

daftarRekening adalah daftar semua rekening yang sudah dibuat dan rekening adalah objek rekening yang sedang diperiksa. getNomorRekening() mengambil nomor rekening tersebut, lalu .equals(nomorTujuan) membandingkan nomor rekening dengan nomor yang dimasukkan pengguna, jika cocok, objek rekening akan disimpan ke akunTujuan. Break untuk menghentikan perulangan karena rekening yang dicari sudah ditemukan.

```
if (akunTujuan == null) {
    System.out.println("Rekening tujuan tidak ditemukan!");
} else if (akunTujuan == akunAktif) {
    System.out.println("Tidak bisa transfer ke rekening sendiri!");
```

akunTujuan == null untuk memastikan apakah nomor rekening tidak ditemukan dalam daftar dan akunTujuan == akunAktif untuk memastikan apakah rekening tujuan transfer aktif atau tidak.

```
} else {
    System.out.print("Masukkan nomor Transfer: ");
    double nominal = input.nextDouble();
    input.nextLine();

    System.out.print("Masukkan PIN Anda: ");
    String pinTransfer = input.nextLine();
```

Double nominal untuk menyimpan jumlah uang yang akan ditransfer, input.nextDouble() untuk membaca nominal berupa angka, input.nextLine() setelahnya mebersihkan sisa karakter baris baru, dan String pin menyimpan PIN yang dimasukkan pengguna.

```
akunAktif.transfer(akunTujuan, nominal, pinTransfer);
```

Memanggil method transfer yang akan menjalankan seluruh proses transfer.

OUTPUT PROGRAM

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 123456
Masukkan Nama Pemilik: syasya
Masukkan Saldo Awal: 500000
Buat PIN (6 digit): 220907

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilih produk: 1
Masukkan Suku Bunga (%): 25
Rekening atas nama syasya berhasil dibuat dengan saldo Rp500000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 234567
Masukkan Nama Pemilik: halwa
Masukkan Saldo Awal: 350000
Buat PIN (6 digit): 070922

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilih produk: 2
Masukkan Batas Overdraft: 200000
Rekening atas nama halwa berhasil dibuat dengan saldo Rp350000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 2
Masukkan nominal setor: 150000
Setor tunai Rp150000.0 berhasil. Saldo saat ini: Rp500000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 3
Masukkan PIN: 50000
Akses Ditolak: PIN yang Anda masukkan salah!

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 4
--- INFO REKENING ---
No. Rekening : 234567
Nama Pemilik : halwa
Saldo Akhir  : Rp500000.0
---------------------

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 5

--- GANTI AKUN ---
Masukkan nomor rekening: 123456
Berhasil berganti ke rekening atas nama syasya

Menu Utama:
1. Buka Rekening Baru
--- GANTI AKUN ---
Masukkan nomor rekening: 123456
Berhasil berganti ke rekening atas nama syasya

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 7
Bunga 25.0% berhasil ditambahkan: Rp125000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 8
Masukkan nomor rekening tujuan: 234567
Masukkan nomor Transfer: 200000
Masukkan PIN Anda: 220907
Tarik tunai Rp200000.0 berhasil. Sisa saldo: Rp425000.0
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp700000.0
Transfer berhasil
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 0
Sistem ditutup. Terima kasih!
```